

Using Interfaces

In Go, a function parameter must declare exactly what type of data it accepts.

- A function like **func Process(x int)** will crash if you try to pass it a **string** or a **float64**.
- But what if you are building a function that needs to process many different types of data, as long as that data shares a common behavior?

This is exactly what **Interfaces** are used for!

1. Generalizing Function Parameters

By using an Interface as a function parameter, you tell Go: "I don't care exactly what concrete struct you hand me, as long as it knows how to perform these specific actions."

- **The Scenario:** You are building an app to calculate if a pool can fit in a backyard. You need to calculate the area (to see if it fits the grass) and the perimeter (to see if you have enough fence).
- **The Problem:** The user might want a rectangular pool, a circular pool, or a triangular pool. You do not want to write three separate functions (**FitRectangle()**, **FitCircle()**, **FitTriangle()**).
- **The Interface Solution:**
 1. You create a **Shape2D** interface that requires an **Area()** method and a **Perimeter()** method.
 2. You write a single function: **func FitInYard(s Shape2D) bool**.
 3. Now, you can pass any custom struct into **FitInYard** (whether it's a Rectangle or a Triangle), and the function will accept it, because it trusts the interface contract!

```
// The function accepts the Interface, not a specific Struct
func FitInYard(s Shape2D) bool {
    // We can safely call these methods because the Interface guarantees they exist
    if s.Area() < 100 && s.Perimeter() < 100 {
        return true
    }
    return false
}
```

2. The Empty Interface (interface{})

What if you want to write a function that is completely unrestricted? You want it to accept literally anything: an integer, a string, a custom struct, or a boolean. You use the **Empty Interface**.

- **What is it?** It is an interface that specifies exactly zero required methods. Because it requires nothing, **every single type in Go automatically satisfies it.**

```
func PrintMe(val interface{}) {
    fmt.Println(val)
}
```

You can pass **PrintMe(5)**, **PrintMe("Hello")**, or **PrintMe(Rectangle{})**, and the compiler will accept all of them perfectly!

Expert Takeaway:

If you have ever used the standard `fmt.Println()` function, you have already used the Empty Interface! If you look at the official Go source code for **Println**, its parameter signature is a **...interface{}**. This means it accepts a variadic number (...) of literally any type (**interface{}**).

Note for modern Go: In Go version 1.18, Google introduced the keyword `any` as an exact alias for `interface{}`. In modern Go codebases, you will almost always see developers write **func PrintMe(val any)** instead, as it is much cleaner to read!